

Methane & CO2 Tracker

Relatório da Fase 9 — Hardening de Produção e Preparação pra Demo Pública

Escopo desta fase

Com o backend já em produção no Render e o frontend no Vercel (Fase 8/Part C), esta fase usou o próprio ambiente de produção como banco de testes real via *smoke_publish_ch4.py* — o que revelou dois bugs de produção que nenhum teste local com Mailpit/Mosquitto teria pego, além de uma lacuna arquitetural real (perda de leitura durante hibernação). Fechou com RBAC mínimo, login de visitante sem credencial exposta, limpeza de dados sintéticos e uma auditoria de segurança/teste completo de encerramento.

Bugs de produção encontrados e corrigidos

- **Timestamp "do futuro" rejeitando toda leitura** — *ReadingIn.time_not_absurd* comparava o timestamp do sensor contra o relógio do próprio processo sem nenhuma tolerância; um container recém-acordado/redeployado no Render tem clock skew real contra o publisher. Corrigido com *FUTURE_TOLERANCE = timedelta(minutes=5)*.
- **Notificação de alerta por e-mail falhando silenciosamente** — *Alert.notified_at* nunca era preenchido. Diagnóstico em duas etapas: primeiro suspeitou-se de timeout curto do SMTP (aumentado de 5s para 15s), mas o traceback completo do Render revelou que o *socket.connect()* pra porta 587 nunca fecha o handshake — bloqueio de saída SMTP comum em provedores de nuvem, não um problema de lentidão. Corrigido trocando SMTP pela **API HTTP do Resend** (porta 443), reaproveitando a mesma credencial já configurada.
- **Leituras perdidas durante hibernação do serviço** — plano free do Render hiberna após 15min sem request HTTP; MQTT nunca acorda o processo, então qualquer leitura publicada nesse intervalo simplesmente desaparecia, sem erro nenhum registrado. Corrigido com sessão MQTT persistente (*clean_session=False + subscribe(qos=1)*) — o HiveMQ Cloud agora enfileira mensagens não entregues e reentrega ao reconectar. Reentregas duplicadas (semântica "at-least-once" do QoS1) são detectadas pela chave primária (*time, sensor_id*) e descartadas silenciosamente (log de debug, não mais um erro genérico).

Confiabilidade do keep-alive

O workflow do GitHub Actions que deveria manter o serviço acordado a cada 10 minutos foi investigado após uma falha real em produção e revelou dois problemas: seu próprio timeout de 30s no curl era mais curto que o cold-start real do Render (35-71s medidos), então uma vez adormecido ele nunca conseguia reacordar o serviço; e o gatilho *schedule* do GitHub Actions não tem SLA de execução, tendo falhado ou sido cancelado em 29 das últimas 30 execuções (confirmado via API pública do GitHub). Substituído por **UptimeRobot** (checagem a cada 5min, alerta automático por e-mail em caso de queda) — infra dedicada a esse propósito, não um cron best-effort.

RBAC mínimo (admin/viewer)

Coluna *role* adicionada a *users* e dependência *require_admin* criada — hoje a API é inteiramente somente leitura (nenhum endpoint de escrita existe ainda), então nada é restrito por ela no momento; existe como guarda-corrado pronta pra qualquer rota de mutação futura já nascer restrita a admin, em vez de depender de alguém lembrar de adicionar o controle na hora. Conta pessoal promovida a *admin*, conta de demo fixada em *viewer*.

Login de visitante sem credencial exposta

O formulário de login continha o e-mail/senha da conta de demo escritos diretamente no código-fonte do frontend — o Vite embute qualquer `VITE_*` como texto literal no bundle JS final, então "esconder" isso da tela não impedia ninguém de extrair a senha inspecionando o bundle. Substituído por **POST /auth/demo-login**: emite token sem receber nenhuma credencial, só funciona se a conta apontada por `DEMO_ACCOUNT_EMAIL` tiver `role=viewer` (travado no servidor, não no cliente), mesmo rate limit de `/auth/login`, expiração mais curta (60min) e log de cada emissão. Verificado: a senha não existe mais em nenhum lugar de `frontend/src` nem no `dist/` gerado.

Limpeza de dados sintéticos

Dados do sensor *smoke-test-ch4* (leituras, alertas) usados só pra validação de pipeline foram removidos do banco de produção antes da divulgação pública, via script dedicado (*cleanup_smoke_test_data.py*) com filtro por igualdade exata de `sensor_id` — sem risco de alcançar os sensores *demo-sensor-** usados por *seed_demo_data.py*.

Teste completo de encerramento

- Suíte automatizada do backend: 7/7 testes independentes de infraestrutura local passando (auth, API REST, relatório de compliance, regra de alerta, detecção de anomalia, roundtrip protobuf, conexão com o banco). 3 testes que dependem de Mailpit/Mosquitto locais (Docker) falharam por timeout de conexão — infraestrutura de dev ausente, não regressão.
- Smoke test completo em produção (após o cold-start real do plano free): leitura, alerta e notificação por e-mail confirmados ponta a ponta mais uma vez após todas as correções desta fase.
- Passagem manual pelo frontend real no navegador, contra a API de produção: login normal, botão "Entrar como Visitante", gráfico de leituras, tabela de eventos de alerta, e geração do PDF de compliance — todos confirmados funcionando (respostas HTTP 200 reais capturadas via rede do navegador), sem nenhum erro de console.

Auditoria de segurança

- Nenhum `.env` real rastreado pelo git (só os `.env.example` com placeholders); a única credencial real que já esteve em texto no histórico de commits foi a senha da conta de demo, já corrigida nesta fase.
- Nenhuma query SQL crua em nenhum lugar do backend — 100% via ORM (SQLAlchemy), risco de injeção de SQL por concatenação de string não se aplica.
- Nenhum uso de `dangerouslySetInnerHTML` ou `eval` no frontend — sem vetor óbvio de XSS via renderização.
- JWT com algoritmo travado explicitamente na decodificação (`algorithms=["HS256"]`) — não vulnerável ao ataque clássico de confusão de algoritmo (`"alg=none"`).
- Rate limiting em duas camadas: 60/min por IP em toda rota (default do *SlowAPIMiddleware*) e 5/min específico em `/auth/login` e `/auth/demo-login`.
- CORS restrito a origem configurada (não wildcard) — confirmado funcionalmente: a origem real de produção no Vercel completou chamadas cross-origin sem nenhum erro de CORS no console do navegador.
- **Achado real (pip-audit)**: *starlette* (dependência transitiva do FastAPI 0.111.1) na versão 0.37.2 tem 9 vulnerabilidades conhecidas no banco de dados consultado, com correção

disponível em versões mais recentes. FastAPI tem versão 0.141.1 disponível (pinado hoje em 0.111.1) — salto grande demais pra aplicar no meio desta auditoria sem teste de regressão dedicado, dado o histórico do projeto com resolução de dependências instável (ver incidente de OOM no build do Render). **Recomendação: tratar como item prioritário da próxima fase**, não decisão tomada aqui.

- *npm audit* nas dependências de produção do frontend: 0 vulnerabilidades encontradas.
- Observação fora do repositório: existe um *Senhas.txt* em texto puro na pasta pai do projeto (fora do controle de versão, não vaza pro git) — recomendação de mover pra um gerenciador de senhas dedicado, especialmente com este projeto prestes a ganhar visibilidade pública.

Arquivos alterados/criados nesta fase

Arquivo	Papel
backend/app/ingestion/schemas.py	Tolerância de 5min no validador de timestamp
backend/app/email_sender.py	Novo — envio de e-mail via API HTTP do Resend
backend/app/alerts/notify.py, anomaly/notify.py	Usam email_sender em vez de smtplib direto
backend/app/ingestion/mqtt_consumer.py	Sessão persistente MQTT (QoS1 + clean_session=False)
backend/app/models/orm.py, alembic 0004	Coluna users.role (admin/viewer)
backend/app/api/deps.py	Dependência require_admin
backend/app/api/routers/auth.py	Novo endpoint POST /auth/demo-login
backend/scripts/cleanup_smoke_test_data.py	Novo — limpeza segura de dados de smoke test
.github/workflows/keep-alive.yml	Removido — substituído por UptimeRobot
frontend/src/components/LoginForm.tsx	Credencial de demo removida, botão de visitante
render.yaml, .env	USE_RESEND_HTTP_API, DEMO_ACCOUNT_EMAIL, DEMO_LOGIN_EXPIRE_MINUT

Não incluído nesta fase (próximos passos)

- Atualizar FastAPI/starlette pra resolver as 9 vulnerabilidades encontradas — precisa de teste de regressão dedicado antes do deploy.
- Campo *is_demo* genérico no banco — hoje a limpeza de dado sintético ainda depende de lista de sensor_id hardcoded por script.
- RBAC ainda sem nenhuma rota de escrita real pra proteger — útil só quando a primeira rota de mutação for criada.
- Rotação do Senhas.txt fora do repositório pra um gerenciador de senhas dedicado.

Gerado automaticamente por backend/scripts/generate_phase9_report.py